Question-1:

🗼 Tower of Hanoi

You are given **N disks** placed on a starting rod. The disks are arranged such that the **smallest disk is on top** and the **largest disk is at the bottom**.

There are three rods:

- **Source Rod** — the rod containing all the disks initially
- **Auxiliary Rod** — an intermediate rod that can be used during the process
- **Target Rod** — the rod where all disks must finally be placed

Your task is to determine the **minimum number of moves** required to transfer all N disks from the source rod to the target rod.

Rules

1. Only **one disk** can be moved at a time.
2. Only the **topmost disk** from a rod can be moved.
3. A disk can only be placed on an **empty rod** or on top of a **larger disk**.
4. All disks must be transferred to the target rod while following the above rules.

Print the moves required to transfer all N disks from the source rod to the target rod.

Solution:

Input:
n = 2

Moves:

Move disk 1 from Rod 1 to Rod 2
Move disk 2 from Rod 1 to Rod 3
Move disk 1 from Rod 2 to Rod 3

Input:
n = 3

Moves:

Move disk 1 from Rod 1 to Rod 3
Move disk 2 from Rod 1 to Rod 2
Move disk 1 from Rod 3 to Rod 2
Move disk 3 from Rod 1 to Rod 3
Move disk 1 from Rod 2 to Rod 1

Move disk 2 from Rod 2 to Rod 3
Move disk 1 from Rod 1 to Rod 3

Therefore, the minimum number of moves is:

7

Input:
n = 0

Output:
0

Explanation:

When there are no disks, no moves are required.

Therefore, the answer is `0`.

Constraints

$0 \leq n \leq 20$

Question-2:

You need to generate all valid N-digit vault codes satisfying:

1. First digit cannot be `0`.
2. No two consecutive digits are the same.
3. Sum of all digits must be exactly `S`.
4. Exactly `K` digits must be even.

Input: `N, S, K`
Output: All valid vault combinations.

Example:
`N = 3, S = 6, K = 2`

Possible output:

114
204
402

Solution:

```
#include <bits/stdc++.h>
using namespace std;

void generateCodes(int N, int S, int K,
                   string &code,
                   int pos,
                   int sum,
                   int evenCount,
                   int prev) {

    // All digits are chosen
    if (pos == N) {
        if (sum == S && evenCount == K) {
            cout << code << endl;
        }
        return;
    }

    // Try every possible digit
    for (int digit = 0; digit <= 9; digit++) {

        // First digit cannot be 0
        if (pos == 0 && digit == 0)
            continue;

        // No two consecutive digits can be same
        if (digit == prev)
            continue;

        // Sum should not exceed S
        if (sum + digit > S)
            continue;

        // Count even digits
        int newEvenCount = evenCount + (digit % 2 == 0);

        // Cannot use more than K even digits
        if (newEvenCount > K)
            continue;

        // Choose
        code.push_back('0' + digit);
```

```
        // Explore
        generateCodes(
            N, S, K,
            code,
            pos + 1,
            sum + digit,
            newEvenCount,
            digit
        );

        // Backtrack
        code.pop_back();
    }
}

int main() {

    int N, S, K;
    cin >> N >> S >> K;

    string code;

    generateCodes(N, S, K, code, 0, 0, 0, -1);

    return 0;
}
```

Question-3:

You are given `N` houses with heights `A[i]`. A flood will rise to a height of `H`.

A house is **safe** if its final height is at least `H`.

You have at most `K` sandbags. If you place a sandbag on a house, its height increases by exactly `X`.

Find the **minimum value of `X`** such that at least `M` houses can be made safe using at most `K` sandbags.

Example

N = 6
A = [2, 5, 1, 7, 3, 4]

H = 6
K = 4
M = 5

For X = 3, only 4 houses can be made safe → **Not possible**.

For X = 4:

2 + 4 = 6 ✅
5 + 4 = 9 ✅
1 ❌
7 ✅
3 + 4 = 7 ✅
4 + 4 = 8 ✅

5 houses are safe using 4 sandbags.

Output:

4

Constraints

$1 \leq N \leq 10^5$
$1 \leq M \leq N$
$0 \leq K \leq N$
$1 \leq A[i], H \leq 10^9$

Input:
N = 5
A = [4, 7, 10, 12, 3]
H = 10
K = 0
M = 2
Answer=0

Input:
N = 5
A = [2, 4, 5, 6, 9]
H = 10
K = 2
M = 1

Answer=1

Solution:

```
bool check(int X) {
    int safe = 0;
    int bags = 0;

    for (int i = 0; i < N; i++) {

        // Already safe
        if (A[i] >= H) {
            safe++;
        }

        // One sandbag can make it safe
        else if (A[i] + X >= H) {
            safe++;
            bags++;
        }

        // Cannot make it safe
        // even with one sandbag
    }

    return safe >= M && bags <= K;
}
```

```
int low = 1;

int high = H;

int ans = H;


while (low <= high) {


    int mid = (low + high) / 2;


    if (check(mid)) {

        ans = mid;

        high = mid - 1;   // try smaller X

    }

    else {

        low = mid + 1;    // need bigger X

    }

}
```